

Actividad 5 · Ae1 — Diseño orientado a objetos de un sistema

Caso: Sistema de gestión de tutorías · GesTutoUEES

Estudiante: Miguel Iván Delgado Anazco

Docente: Ph.D. Jaime Paul Sayago Heredia

Fecha: Agosto 2026

Pregunta orientadora: ¿las clases, relaciones y dependencias de esta propuesta reflejan responsabilidades claras y permiten que el sistema evolucione con un impacto controlado? Esa es la vara con la que medí cada decisión de este diseño.

2 Análisis del dominio

Del caso saqué los objetos que de verdad importan y qué hace cada uno. No puse clases por rellenar: cada una responde a una necesidad del enunciado.

Elemento	Responsabilidad	Reglas / colaboraciones
Usuario (abstracta)	Lo común de cualquier persona del sistema (id, nombre, correo).	Nadie se registra como "usuario genérico": es base de Estudiante y Docente.
Estudiante	Solicita y reserva tutorías.	Hereda de Usuario. Aparece en la Reserva.
Docente	Publica y administra sus horarios.	Hereda de Usuario. Un docente tiene muchos horarios.
Horario	Un bloque de tiempo disponible.	Protege su disponibilidad: ocupar() falla si ya está ocupado (anti doble reserva).
Reserva	El encuentro entre estudiante y horario.	Ciclo de vida propio: confirmar, cancelar (libera el horario), reprogramar.
ServicioReservas	Orquesta la lógica de negocio.	Coordina horarios, reservas, persistencia y avisos.
Notificador / Repositorio	Abstracciones del aviso y de la persistencia.	El dominio depende de estas interfaces, no de tecnologías concretas.

3 Diseño orientado a objetos

Herencia solo donde es verdad. Estudiante y Docente heredan de Usuario porque *son* usuarios, no para reutilizar código. **Composición para lo demás:** una Reserva *tiene* un Estudiante y un Horario; heredar ahí sería forzar la relación.

El estado se protege, no se expone. El Horario no deja cambiar disponible desde fuera: solo ocupar() y liberar(). Así la regla anti doble reserva vive en un solo lugar y no se puede saltar por accidente. Lo mismo con Reserva: sus métodos validan el estado antes de cambiarlo, para que nadie la deje inconsistente.

4 Cohesión y acoplamiento

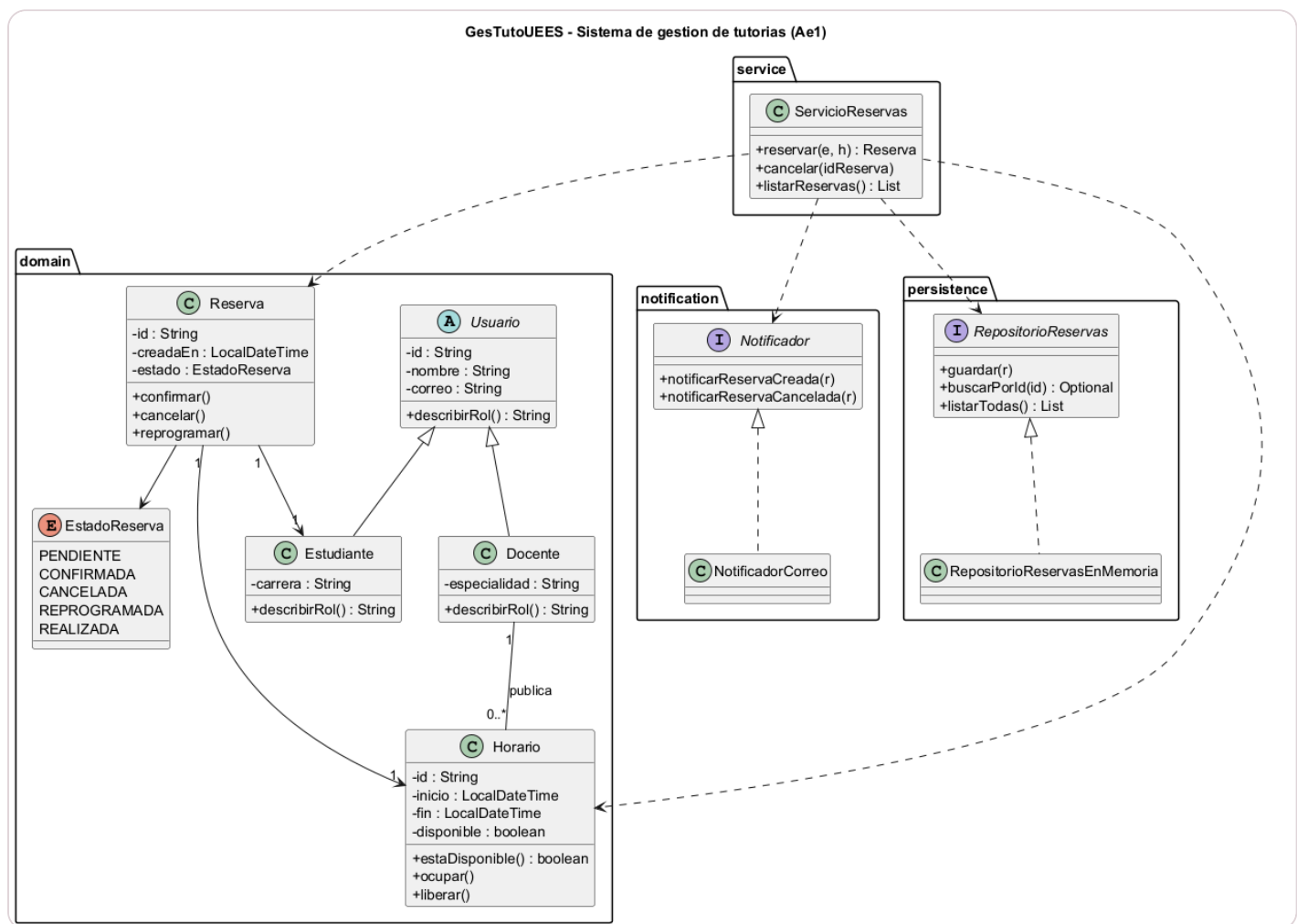
Cohesión alta. Cada clase hace una cosa: `Horario` maneja su tiempo, `Reserva` su ciclo de vida, `ServicioReservas` coordina. Si algo falla en la reserva, sé exactamente a qué clase ir. Eso es cohesión.

Acoplamiento controlado con abstracciones. La dependencia que más riesgo tenía era la del aviso y la de la persistencia: si `ServicioReservas` hubiera creado un `NotificadorCorreo` por dentro, cambiar de correo a SMS habría tocado el dominio. Lo aislé con las interfaces `Notificador` y `RepositorioReservas`. Cambiar la tecnología de persistencia o de notificación hoy significa crear una clase nueva, no modificar el dominio.

5 Principios SOLID

- **DIP (Dependency Inversion):** `ServicioReservas` depende de las abstracciones `Notificador` y `RepositorioReservas`, no de implementaciones concretas. Se inyectan por el constructor (en `Main`). Problema que evita: que el dominio quede atado a una tecnología. Evidencia: el constructor del servicio recibe las interfaces.
- **SRP (Single Responsibility):** cada clase tiene una razón para cambiar. `Reserva` cambia si cambian las reglas de la reserva; `NotificadorCorreo` cambia si cambia el canal de aviso. No hay una clase que hace de todo.
- **OCP (Open/Closed):** se puede agregar un `NotificadorSms` o un `RepositorioReservasPostgres` sin tocar el código que ya funciona: se extiende, no se modifica.

6 Diagrama UML de clases



Modelo de clases: herencia, realización de interfaces, asociaciones y dependencias (DIP). El UML y el código cuentan la misma historia.

7 Conclusiones

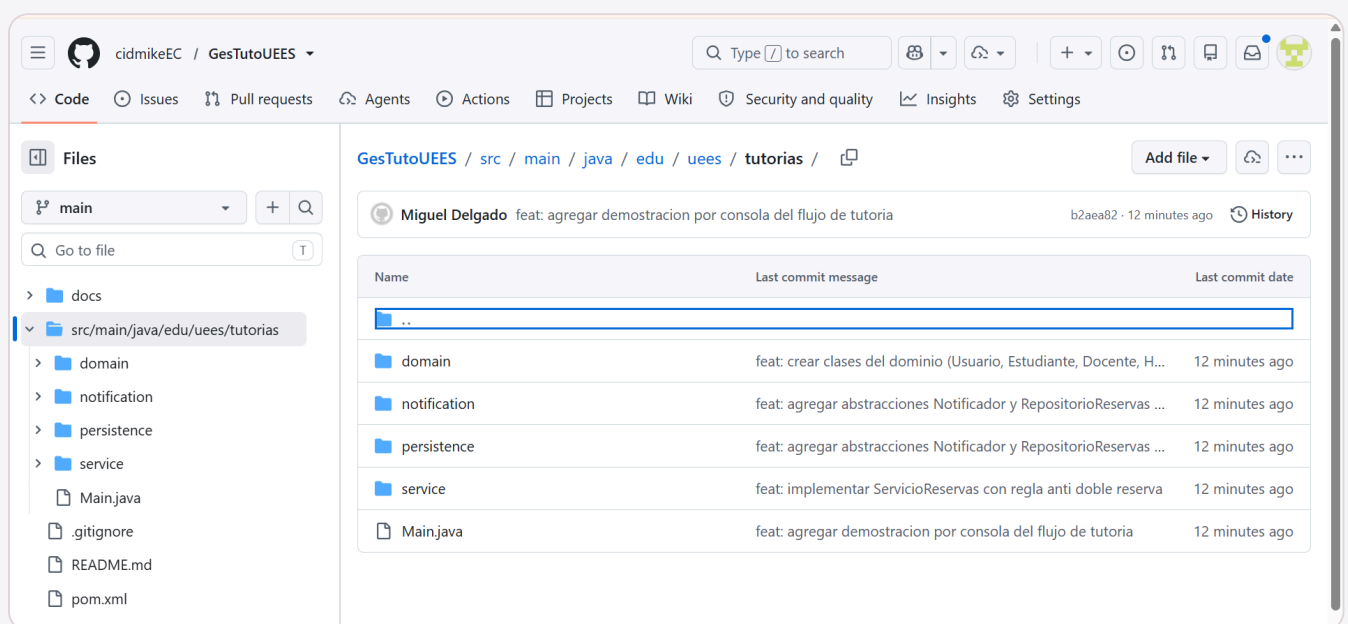
Diseñar antes de programar se nota. Separar el dominio de las tecnologías (avisos y persistencia) con dos interfaces hizo que el sistema quede abierto a cambios sin romperse. La regla más importante —que un horario ocupado no se reserve dos veces— quedó protegida dentro del propio objeto, no repartida por el código. Un sistema no es sólido por lo complejo que sea, sino por lo barato que le resulte el próximo cambio.

8 Repositorio

GitHub (público):

<https://github.com/cidmikeEC/GesTutoUEES>

El historial de commits muestra la evolución: estructura Maven → clases del dominio → abstracciones (DIP) → servicio → demostración → UML → README. Compila con `mvn clean compile`.



Name	Last commit message	Last commit date
..		
domain	feat: crear clases del dominio (Usuario, Estudiante, Docente, H...	12 minutes ago
notification	feat: agregar abstracciones Notificador y RepositorioReservas ...	12 minutes ago
persistence	feat: agregar abstracciones Notificador y RepositorioReservas ...	12 minutes ago
service	feat: implementar ServicioReservas con regla anti doble reserva	12 minutes ago
Main.java	feat: agregar demostracion por consola del flujo de tutoria	12 minutes ago

Repositorio publico en GitHub: estructura de paquetes (domain, service, notification, persistence), historial de commits y README.

Declaración de uso de IA: durante el desarrollo de esta actividad utilicé herramientas de inteligencia artificial como apoyo para estructurar el proyecto y redactar borradores. Verifiqué, adapté y comprendí el código y las decisiones aquí presentadas, y puedo explicarlas y justificarlas.